

Boa noite! Entendi sua solicitação para desenvolver um sistema de vendas inspirado no projeto do link (que é um sistema front-end em JavaScript com armazenamento local, focado em vendas para armarinho, com funcionalidades como carrinho, clientes, produtos, cupons fiscais/não fiscais, backups e relatórios). Como você pediu para preservar a linguagem Python, vou propor uma implementação em Python usando o framework Flask (simples e leve para um sistema web), com banco de dados SQLite (para manter a simplicidade e portabilidade, similar ao armazenamento local do original). Vou manter a "cronologia" lógica do sistema original: login com níveis de acesso (operador/admin), telas de vendas, cadastros, cupons, backups e relatórios, mas adaptando para backend Python.

Vou estruturar a resposta assim:

- **Plano geral do sistema:** Estrutura e como implementar as funcionalidades novas.
- **Ideias para integrações:** Especificamente para APIs de pagamentos (PIX, cartões) e envio via WhatsApp.
- **Código base em Python:** Um esqueleto funcional para você rodar e expandir. Usei Flask, SQLAlchemy para BD, e bibliotecas simples. Você pode rodar isso localmente com `pip install flask flask-sqlalchemy` e executar o app.

Plano Geral do Sistema

- **Tecnologias:** Python 3 (Flask para web app), SQLite (BD), HTML/CSS/JS simples no front-end (inspirado no original, com modais e atalhos). Para backups, usarei exportação JSON automática/manual, similar ao original.

- **Estrutura:**

- **Models (BD):** Tabelas para Usuários (admin/operador com senhas), Clientes (nome, CPF/CNPJ, telefone, email, endereço), Produtos (código, descrição, preço, estoque), Vendas (ID cupom, data/hora, cliente, itens, total, status: emitido/cancelado), Cupons (detalhes do cupom ligado à venda).
- **Rotas/Views:** Login, dashboard operador (vendas/carrinho), admin (cadastros, backups, ajustes API, relatórios leitura Z).
- **Fluxo de Vendas:** Adicionar itens ao carrinho, selecionar cliente, finalizar (gerar cupom), opção de imprimir/enviar WhatsApp.
- **Novas Funcionalidades:**
 - **Pesquisa de Vendas:** Campo de busca no dashboard de vendas/admin para filtrar por número de cupom (ID), cliente (nome/CPF), ou horário (data/hora). Ao encontrar, exibir segunda via do cupom (visualização/impressão/envio).
 - **Cupom:** Template fixo com apenas o nome da empresa (cadastrado no admin, com CPF/CNPJ). Cupom fiscal/não fiscal gerado em PDF/HTML. Sempre enviar cópia ao admin via WhatsApp (independente de impressão ou escolha do cliente).
 - **Cancelamento:** Requer senha admin. Ao cancelar, atualiza status no BD, gera nova versão do cupom com carimbo "CANCELADO" e reenvia ao admin via WhatsApp.
 - **Botão Admin para Ajustes API:** Na área admin, formulário para configurar chaves API de pagamentos (ex: chave Mercado Pago). Salva no BD ou arquivo config.
 - **Envio WhatsApp:** Opção no final da venda para cliente (número cadastrado). Sempre envia ao admin (número fixo cadastrado no config).
- **Segurança:** Senhas hashadas (use `werkzeug.security`), confirmação admin para ações sensíveis.
- **Backups:** Automático diário (use scheduler como `schedule lib`), exporta BD para JSON.

- **Execução:** App web local (pode deployar em Heroku/Vercel). Para impressão, use `JS window.print()`.

Ideias para Integrações

- **APIs de Pagamentos (PIX, Cartões Crédito/Débito):**
 - **Mercado Pago:** Excelente para Brasil, suporta PIX, cartões e boleto. Ideia: No botão admin, campos para inserir `access_token` e `public_key`. Na finalização de venda, gere QR Code PIX ou link de pagamento via API. Biblioteca Python: `mercadopago` (`pip install`). Exemplo: Crie uma rota `/pagamento` que chama `sdk.payment().create()` para PIX/cartão.
 - **PagSeguro:** Similar, com sandbox para testes. Ideia: Configure no admin webhook para notificações de pagamento aprovado. Biblioteca: `pagseguro-python`.
 - **Stripe:** Se internacional, mas para BR use adaptações. Ideia: Botão "Testar API" no admin para validar chaves.
 - **Dica Geral:** Comece com sandbox. No código, ao finalizar venda, se API configurada, integre; senão, fallback para pagamento manual (como no original). Para cartões, use formulário seguro (não armazene dados sensíveis).
- **Envio via WhatsApp:**
 - **Twilio:** Fácil integração. Ideia: No admin, configure SID, token e número Twilio. Biblioteca: `twilio` (`pip install`). Envie cupom como mensagem com texto/PDF anexo. Exemplo: `client.messages.create(to=numero_cliente, from_=numero_twilio, body=cupom_texto, media_url=pdf_url)`. Sempre envie ao admin (número fixo).
 - **WhatsApp Business API:** Oficial, mas requer aprovação. Ideia: Use via provedores como MessageBird ou diretamente. Para envio automático: Após emitir cupom, chame API para enviar template com cupom.
 - **Dicas:** Gere PDF do cupom com `reportlab` (`pip install`). Para cancelado, adicione watermark "CANCELADO". Sempre envie ao admin, mesmo se cliente recuse ou não imprima.

- **Outras Ideias:**

- Notificações: Integre email (smtplib) como fallback para WhatsApp.
- Relatórios: Expanda leitura Z para exportar CSV/PDF com filtros por data.
- Mobile: Torne responsivo para uso em tablet/caixa registradora.
- Testes: Use unittest para validar fluxos de venda/cancelamento.

Código Base em Python (Flask)

Aqui vai um esqueleto funcional. Salve como `app.py` , crie pastas `templates` (para HTML) e rode `python app.py` . Expanda com as ideias acima.

Python

```
from flask import Flask, render_template, request, redirect, url_for, session, flash
from flask_sqlalchemy import SQLAlchemy
from datetime import datetime
import json
import os
# Para hashes de senha: from werkzeug.security import generate_password_hash,
check_password_hash
# Para PDF: from reportlab.pdfgen import canvas (instale reportlab)
# Para scheduler backups: import schedule, time (instale schedule)

app = Flask(__name__)
app.config['SECRET_KEY'] = 'sua_chave_secreta'
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///sistema_vendas.db'
db = SQLAlchemy(app)

# Models
class Usuario(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(50), unique=True)
    password = db.Column(db.String(100)) # Hasheada
    role = db.Column(db.String(20)) # 'admin' ou 'operador'

class Cliente(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    nome = db.Column(db.String(100))

    documento = db.Column(db.String(20)) # CPF/CNPJ
```



```
telefone = db.Column(db.String(20))
email = db.Column(db.String(100))
```

```
class Produto(db.Model):
```

```
    id = db.Column(db.Integer, primary_key=True)
    codigo = db.Column(db.String(50))
    descricao = db.Column(db.String(100))
    preco = db.Column(db.Float)
    estoque = db.Column(db.Integer)
```

```
class Venda(db.Model):
```

```
    id = db.Column(db.Integer, primary_key=True) # Número de cupom
    data_hora = db.Column(db.DateTime, default=datetime.utcnow)
    cliente_id = db.Column(db.Integer, db.ForeignKey('cliente.id'))
    itens = db.Column(db.Text) # JSON de itens
    total = db.Column(db.Float)
    status = db.Column(db.String(20), default='emitido') # 'emitido' ou 'cancelado'
```

```
# Config Empresa (salva em BD ou arquivo)
```

```
class Config(db.Model):
```

```
    id = db.Column(db.Integer, primary_key=True)
    nome_empresa = db.Column(db.String(100))
    documento_empresa = db.Column(db.String(20)) # CPF/CNPJ
    numero_admin_whatsapp = db.Column(db.String(20))
    api_pagamento = db.Column(db.Text) # JSON com chaves API
```

```
# Criar BD
```

```
with app.app_context():
```

```
    db.create_all()
```

```
    # Usuários default (hasheie as senhas em produção)
```

```
    if not Usuario.query.first():
```

```
        db.session.add(Usuario(username='admin', password='admin', role='admin'))
```

```
        db.session.add(Usuario(username='operador', password='operador123', role='operador'))
```

```
        db.session.add(Config(id=1, nome_empresa='Sua Empresa',
```

```
documento_empresa='00.000.000/0001-00', numero_admin_whatsapp='+5511999999999'))
```

```
        db.session.commit()
```

```
# Rotas
```

```
@app.route('/')
```

```
def index():
```

```
    return redirect(url_for('login'))
```

```

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        user = Usuario.query.filter_by(username=request.form['username']).first()
        if user and user.password == request.form['password']: # Use check_password_hash em prod
            session['user_id'] = user.id
            session['role'] = user.role
            return redirect(url_for('dashboard'))
        flash("Login inválido")
    return render_template('login.html') # Crie HTML simples

```

```

@app.route('/dashboard')
def dashboard():
    if 'user_id' not in session: return redirect(url_for('login'))
    return render_template('dashboard.html', role=session['role']) # Telas de vendas/admin

```

Exemplo: Finalizar Venda e Gerar Cupom

```

@app.route('/venda/finalizar', methods=['POST'])
def finalizar_venda():
    # Lógica: Salvar venda, gerar cupom PDF/HTML com nome empresa só
    nova_venda = Venda(cliente_id=request.form['cliente_id'], itens=json.dumps(request.form['itens']),
total=float(request.form['total']))
    db.session.add(nova_venda)
    db.session.commit()
    cupom_id = nova_venda.id
    # Gerar cupom (ex: texto simples por agora)
    config = Config.query.first()
    cupom_texto = f"Cupom {cupom_id} - {config.nome_empresa}
({config.documento_empresa})\nData: {nova_venda.data_hora}\nItens: {nova_venda.itens}\nTotal:
R${nova_venda.total}"
    # Envio WhatsApp (integre Twilio aqui)
    # Sempre envia ao admin
    # Se cliente quiser: envie ao telefone do cliente

    # Opção impressão: Retorne HTML com print JS
    flash("Venda finalizada! Cupom enviado ao admin.")
    return redirect(url_for('dashboard'))

```

Pesquisa Vendas

```

@app.route('/venda/pesquisar')
def pesquisar_venda():
    query = request.args.get('query') # Por cupom, cliente ou data
    vendas = Venda.query.filter( # Lógica de filtro: like para nome, == para id, between para data

```

```
db.or_(Venda.id == query, Cliente.nome.like(f'%{query}%'), Venda.data_hora.like(f'%{query}%'))
).join(Cliente).all()
return render_template('pesquisa.html', vendas=vendas) # Exiba segunda via
```

Cancelar Cupom

```
@app.route('/venda/cancelar/<int:id>', methods=['POST'])
def cancelar_venda(id):
    if session['role'] != 'admin' or request.form['senha'] != 'admin': # Valide senha
        flash('Acesso negado')
        return redirect(url_for('dashboard'))
    venda = Venda.query.get(id)
    venda.status = 'cancelado'
    db.session.commit()
    # Re-gerar cupom com "CANCELADO" e reenviar ao admin via WhatsApp
    flash('Cupom cancelado e reenviado ao admin.')
    return redirect(url_for('dashboard'))
```

Admin: Ajustar API Pagamentos

```
@app.route('/admin/ajustar_api', methods=['GET', 'POST'])
def ajustar_api():
    if session['role'] != 'admin': return redirect(url_for('login'))
    config = Config.query.first()
    if request.method == 'POST':
        api_data = json.dumps({'mercado_pago_token': request.form['token'], ...}) # Campos para
chaves
        config.api_pagamento = api_data
        db.session.commit()
        flash('API ajustada!')
    return render_template('ajustar_api.html', config=config)
```

Backup (exemplo manual)

```
@app.route('/admin/backup')
def backup():
    data = [row.__dict__ for row in Venda.query.all()] # Expanda para todas tabelas
    with open('backup.json', 'w') as f:
        json.dump(data, f)
    flash('Backup criado!')
    return redirect(url_for('dashboard'))
```

Para backups automáticos: Use schedule in a thread

```
# def run_scheduler():
#     schedule.every().day.at("23:00").do(backup)
```

```
# while True: schedule.run_pending(); time.sleep(1)
# Inicie em thread no app.run

if __name__ == '__main__':
    app.run(debug=True)
```

Para templates HTML: Crie arquivos em `templates/` com forms simples, inspirados no original (modais para cadastros, carrinho). Adicione JS para atalhos (F3 finalizar, etc.).

Expanda isso: Integre as libs de pagamento/WhatsApp nos pontos comentados. Se precisar de ajuda em partes específicas (ex: código para PDF ou Twilio), me avise! Teste localmente e me conte como foi.